

As we compare MOVE-TO-FRONT and FORESEE searching and modifying their lists, we'll think about MOVE-TO-FRONT executing on its list for the i th time and then FORESEE executing on its list for the i th time. After MOVE-TO-FRONT has executed for the i th time and before FORESEE has executed for the i th time, we'll compare $I(L_{i-1}^M, L_{i-1}^F)$ (the inversion count immediately before the i th call of MOVE-TO-FRONT) with $I(L_i^M, L_{i-1}^F)$ (the inversion count after the i th call of MOVE-TO-FRONT but before the i th call of FORESEE). We'll concern ourselves later with what FORESEE does.

Let us analyze what happens to the inversion count after executing the i th call of MOVE-TO-FRONT, and suppose that it searches for element x . More precisely, we'll compute $I(L_i^M, L_{i-1}^F) - I(L_{i-1}^M, L_{i-1}^F)$, the change in the inversion count, which gives a rough idea of how much MOVE-TO-FRONT's list becomes more or less like FORESEE's list. After searching, MOVE-TO-FRONT performs a series of swaps with each of the elements on the list L_{i-1}^M that precedes x . Using the notation above, the number of such swaps is $|BB| + |BA|$. Bearing in mind that the list L_{i-1}^F has yet to be changed by the i th call of FORESEE, let's see how the inversion count changes.

Consider a swap with an element $y \in BB$. Before the swap, y precedes x in both L_{i-1}^M and L_{i-1}^F . After the swap, x precedes y in L_i^M , and L_{i-1}^F does not change. Therefore, the inversion count increases by 1 for each element in BB . Now consider a swap with an element $z \in BA$. Before the swap, z precedes x in L_{i-1}^M but x precedes z in L_{i-1}^F . After the swap, x precedes z in both lists. Therefore, the inversion count decreases by 1 for each element in BA . Thus altogether, the inversion count increases by

$$I(L_i^M, L_{i-1}^F) - I(L_{i-1}^M, L_{i-1}^F) = |BB| - |BA|. \quad (27.7)$$

We have laid the groundwork needed to analyze MOVE-TO-FRONT.

Theorem 27.1

Algorithm MOVE-TO-FRONT has a competitive ratio of 4.

Proof The proof uses a potential function, as described in Chapter 16 on amortized analysis. The value Φ_i of the potential function after the i th calls of MOVE-TO-FRONT and FORESEE depends on the inversion count:

$$\Phi_i = 2I(L_i^M, L_i^F).$$

(Intuitively, the factor of 2 embodies the notion that each inversion represents a cost of 2 for MOVE-TO-FRONT relative to FORESEE: 1 for searching and 1 for swapping.) By equation (27.7), after the i th call of MOVE-TO-FRONT, but before the i th call of FORESEE, the potential increases by $2(|BB| - |BA|)$. Since the inversion count of the two lists is nonnegative, we have $\Phi_i \geq 0$ for all $i \geq 0$.

Assuming that MOVE-TO-FRONT and FORESEE start with the same list, the initial potential Φ_0 is 0, so that $\Phi_i \geq \Phi_0$ for all i .

Drawing from equation (16.2) on page 456, the amortized cost $\hat{c}_i^M$ of the i th MOVE-TO-FRONT operation is

$$\hat{c}_i^M = c_i^M + \Phi_i - \Phi_{i-1},$$

where c_i^M , the actual cost of the i th MOVE-TO-FRONT operation, is given by equation (27.3):

$$c_i^M = 2r_{L_{i-1}^M}(x) - 1.$$

Now, let's consider the potential change $\Phi_i - \Phi_{i-1}$. Since both L^M and L^F change, let's consider the changes to one list at a time. Recall that when MOVE-TO-FRONT moves element x to the front, it increases the potential by exactly $2(|BB| - |BA|)$. We now consider how the optimal algorithm FORESEE changes its list L^F : it performs t_i swaps. Each swap performed by FORESEE either increases or decreases the potential by 2, and thus the increase in potential by FORESEE in the i th call can be at most $2t_i$. We therefore have

$$\begin{aligned} \hat{c}_i^M &= c_i^M + \Phi_i - \Phi_{i-1} \\ &\leq 2r_{L_{i-1}^M}(x) - 1 + 2(|BB| - |BA| + t_i) \\ &= 2r_{L_{i-1}^M}(x) - 1 + 2(|BB| - (r_{L_{i-1}^M}(x) - 1 - |BB|) + t_i) \\ &\quad \text{(by equation (27.5))} \\ &= 4|BB| + 1 + 2t_i \\ &\leq 4|BB| + 4|AB| + 4 + 4t_i && \text{(increasing some terms)} \\ &= 4(|BB| + |AB| + 1 + t_i) \\ &= 4(r_{L_{i-1}^F}(x) + t_i) && \text{(by equation (27.6))} \\ &= 4c_i^F && \text{(by equation (27.4))} . \end{aligned} \tag{27.8}$$

We now finish the proof as in Chapter 16 by showing that the total amortized cost provides an upper bound on the total actual cost, because the initial potential function is 0 and the potential function is always nonnegative. By equation (16.3) on page 456, for any sequence of m MOVE-TO-FRONT operations, we have

$$\begin{aligned} \sum_{i=1}^m \hat{c}_i^M &= \sum_{i=1}^m c_i^M + \Phi_m - \Phi_0 \\ &\geq \sum_{i=1}^m c_i^M && \text{(because } \Phi_m \geq \Phi_0 \text{)} . \end{aligned} \tag{27.9}$$

Therefore, we have

$$\begin{aligned}
 \sum_{i=1}^m c_i^M &\leq \sum_{i=1}^m \hat{c}_i^M && \text{(by equation (27.9))} \\
 &\leq \sum_{i=1}^m 4c_i^F && \text{(by equation (27.8))} \\
 &= 4 \sum_{i=1}^m c_i^F .
 \end{aligned}$$

Thus the total cost of the m MOVE-TO-FRONT operations is at most 4 times the total cost of the m FORESEE operations, so MOVE-TO-FRONT is 4-competitive. ■

Isn't it amazing that we can compare MOVE-TO-FRONT with the optimal algorithm FORESEE when we have no idea of the swaps that FORESEE makes? We were able to relate the performance of MOVE-TO-FRONT to the optimal algorithm by capturing how particular properties (swaps in this case) must evolve relative to the optimal algorithm, without actually knowing the optimal algorithm.

The online algorithm MOVE-TO-FRONT has a competitive ratio of 4: on any input sequence, it incurs a cost at most 4 times that of any other algorithm. On a particular input sequence, it could cost much less than 4 times the optimal algorithm, perhaps even matching the optimal algorithm.

Exercises

27.2-1

You are given a set $S = \{x_1, x_2, \dots, x_n\}$ of n elements, and you wish to make a static list L (no rearranging once the list is created) containing the elements of S that is good for searching. Suppose that you have a probability distribution, where $p(x_i)$ is the probability that a given search searches for element x_i . Argue that the expected cost for m searches is

$$m \sum_{i=1}^n p(x_i) \cdot r_L(x_i) .$$

Prove that this sum is minimized when the elements of L are sorted in decreasing order with respect to $p(x_i)$.

27.2-2

Professor Carnac claims that since FORESEE is an optimal algorithm that knows the future, then at each step it must incur no more cost than MOVE-TO-FRONT. Either prove that Professor Carnac is correct or provide a counterexample.

27.2-3

Another way to maintain a linked list for efficient searching is for each element to maintain a *frequency count*: the number of times that the element has been searched for. The idea is to rearrange list elements after searches so that the list is always sorted by decreasing frequency count, from largest to smallest. Either show that this algorithm is $O(1)$ -competitive, or prove that it is not.

27.2-4

The model in this section charged a cost of 1 for each swap. We can consider an alternative cost model in which, after accessing x , you can move x anywhere earlier in the list, and there is no cost for doing so. The only cost is the cost of the actual accesses. Show that MOVE-TO-FRONT is 2-competitive in this cost model, assuming that the number requests is sufficiently large. (*Hint*: Use the potential function $\Phi_i = I(L_i^M, L_i^F)$.)

27.3 Online caching

In Section 15.4, we studied the caching problem, in which *blocks* of data from the main memory of a computer are stored in the *cache*: a small but faster memory. In that section, we studied the offline version of the problem, in which we assumed that we knew the sequence of memory requests in advance, and we designed an algorithm to minimize the number of cache misses. In almost all computer systems, caching is, in fact, an online problem. We do not generally know the series of cache requests in advance; they are presented to the algorithm only as the requests for blocks are actually made. To gain a better understanding of this more realistic scenario, we analyze online algorithms for caching. We will first see that all deterministic online algorithms for caching have a lower bound of $\Omega(k)$ for the competitive ratio, where k is the size of the cache. We will then present an algorithm with a competitive ratio of $\Theta(n)$, where the input size is n , and one with a competitive ratio of $O(k)$, which matches the lower bound. We will end by showing how to use randomization to design an algorithm with a much better competitive ratio of $\Theta(\lg k)$. We will also discuss the assumptions that underlie randomized online algorithms, via the notion of an adversary, such as we saw in Chapter 11 and will see in Chapter 31.

You can find the terminology used to describe the caching problem in Section 15.4, which you might wish to review before proceeding.

27.3.1 Deterministic caching algorithms

In the caching problem, the input comprises a sequence of n memory requests, for data in blocks $b_1, b_2, \dots, b_n$, in that order. The blocks requested are not necessarily distinct: each block may appear multiple times within the request sequence. After block b_i is requested, it resides in a cache that can hold up to k blocks, where k is a fixed cache size. We assume that $n > k$, since otherwise we are assured that the cache can hold all the requested blocks at once. When a block b_i is requested, if it is already in the cache, then a *cache hit* occurs and the cache remains unchanged. If b_i is not in the cache, then a *cache miss* occurs. If the cache contains fewer than k blocks upon a cache miss, block b_i is placed into the cache, which now contains one block more than before. If a cache miss occurs with an already full cache, however, some block must be evicted from the cache before b_i can enter. Thus, a caching algorithm must decide which block to evict from the cache upon a cache miss when the cache is full. The goal is to minimize the number of cache misses over the entire request sequence. The caching algorithms considered in this chapter differ only in which block they decide to evict upon a cache miss. We do not consider abilities such as prefetching, in which a block is brought into the cache before an upcoming request in order to avert a future cache miss.

There are many online caching policies to determine which block to evict, including the following:

- First-in, first-out (FIFO): evict the block that has been in the cache the longest time.
- Last-in, first-out (LIFO): evict the block that has been in the cache the shortest time.
- Least Recently Used (LRU): evict the block whose last use is furthest in the past.
- Least Frequently Used (LFU): evict the block that has been accessed the fewest times, breaking ties by choosing the block that has been in the cache the longest.

To analyze these algorithms, we assume that the cache starts out empty, so that no evictions occur during the first k requests. We wish to compare the performance of an online algorithm to an optimal offline algorithm that knows the future requests. As we will soon see, all these deterministic online algorithms have a lower bound of $\Omega(k)$ for their competitive ratio. Some deterministic algorithms also have a competitive ratio with an $O(k)$ upper bound, but some other deterministic algorithms are considerably worse, having a competitive ratio of $\Theta(n/k)$.

We now proceed to analyze the LIFO and LRU policies. In addition to assuming that $n > k$, we will assume that at least k distinct blocks are requested. Otherwise, the cache never fills up and no blocks are evicted, so that all algorithms exhibit the same behavior. We begin by showing that LIFO has a large competitive ratio.

Theorem 27.2

LIFO has a competitive ratio of $\Theta(n/k)$ for the online caching problem with n requests and a cache of size k .

Proof We first show a lower bound of $\Omega(n/k)$. Suppose that the input consists of $k + 1$ blocks, numbered $1, 2, \dots, k + 1$, and the request sequence is

$$1, 2, 3, 4, \dots, k, k + 1, k, k + 1, k, k + 1, \dots,$$

where after the initial $1, 2, \dots, k, k + 1$, the remainder of the sequence alternates between k and $k + 1$, with a total of n requests. The sequence ends on block k if n and k are either both even or both odd, and otherwise, the sequence ends on block $k + 1$. That is, $b_i = i$ for $i = 1, 2, \dots, k - 1$, $b_i = k + 1$ for $i = k + 1, k + 3, \dots$ and $b_i = k$ for $i = k, k + 2, \dots$. How many blocks does LIFO evict? After the first k requests (which are considered to be cache misses), the cache is filled with blocks $1, 2, \dots, k$. The $(k + 1)$ st request, which is for block $k + 1$, causes block k to be evicted. The $(k + 2)$ nd request, which is for block k , forces block $k + 1$ to be evicted, since that block was just placed into the cache. This behavior continues, alternately evicting blocks k and $k + 1$ for the remaining requests. LIFO, therefore, suffers a cache miss on every one of the n requests.

The optimal offline algorithm knows the entire sequence of requests in advance. Upon the first request of block $k + 1$, it just evicts any block except block k , and then it never evicts another block. Thus, the optimal offline algorithm evicts only once. Since the first k requests are considered cache misses, the total number of cache misses is $k + 1$. The competitive ratio, therefore, is $n/(k + 1)$, or $\Omega(n/k)$.

For the upper bound, observe that on any input of size n , any caching algorithm incurs at most n cache misses. Because the input contains at least k distinct blocks, any caching algorithm, including the optimal offline algorithm, must incur at least k cache misses. Therefore, LIFO has a competitive ratio of $O(n/k)$. ■

We call such a competitive ratio *unbounded*, because it grows with the input size. Exercise 27.3-2 asks you to show that LFU also has an unbounded competitive ratio.

FIFO and LRU have a much better competitive ratio of $\Theta(k)$. There is a big difference between competitive ratios of $\Theta(n/k)$ and $\Theta(k)$. The cache size k is independent of the input sequence and does not grow as more requests arrive over time. A competitive ratio that depends on n , on the other hand, does grow with the size of the input sequence and thus can get quite large. It is preferable to use an algorithm with a competitive ratio that does not grow with the input sequence's size, when possible.

We now show that LRU has a competitive ratio of $\Theta(k)$, first showing the upper bound.

Theorem 27.3

LRU has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k .

Proof To analyze LRU, we will divide the sequence of requests into *epochs*. Epoch 1 begins with the first request. Epoch i , for $i > 1$, begins upon encountering the $(k + 1)$ st distinct request since the beginning of epoch $i - 1$. Consider the following example of requests with $k = 3$:

$$1, 2, 1, 5, 4, 4, 1, 2, 4, 2, 3, 4, 5, 2, 2, 1, 2, 2. \quad (27.10)$$

The first $k = 3$ distinct requests are for blocks 1, 2 and 5, so epoch 2 begins with the first request for block 4. In epoch 2, the first 3 distinct requests are for blocks 4, 1, and 2. Requests for these blocks recur until the request for block 3, and with this request epoch 3 begins. Thus, this example has four epochs:

$$1, 2, 1, 5 \quad 4, 4, 1, 2, 4, 2 \quad 3, 4, 5 \quad 2, 2, 1, 2, 2. \quad (27.11)$$

Now we consider the behavior of LRU. In each epoch, the first time a request for a particular block appears, it may cause a cache miss, but subsequent requests for that block within the epoch cannot cause a cache miss, since the block is now one of the k most recently used. For example, in epoch 2, the first request for block 4 causes a cache miss, but the subsequent requests for block 4 do not. (Exercise 27.3-1 asks you to show the contents of the cache after each request.) In epoch 3, requests for blocks 3 and 5 cause cache misses, but the request for block 4 does not, because it was recently accessed in epoch 2. Since only the first request for a block within an epoch can cause a cache miss and the cache holds k blocks, each epoch incurs at most k cache misses.

Now consider the behavior of the optimal algorithm. The first request in each epoch must cause a cache miss, even for an optimal algorithm. The miss occurs because, by the definition of an epoch, there *must* have been k other blocks accessed since the last access to this block.

Since, for each epoch, the optimal algorithm incurs at least one miss and LRU incurs at most k , the competitive ratio is at most $k/1 = O(k)$. ■

Exercise 27.3-3 asks you to show that FIFO also has a competitive ratio of $O(k)$.

We could show lower bounds of $\Omega(k)$ on LRU and FIFO, but in fact, we can make a much stronger statement: *any* deterministic online caching algorithm must have a competitive ratio of $\Omega(k)$. The proof relies on an adversary who knows the online algorithm being used and can tailor the future requests to cause the online algorithm to incur more cache misses than the optimal offline algorithm.

Consider a scenario in which the cache has size k and the set of possible blocks to request is $\{1, 2, \dots, k + 1\}$. The first k requests are for blocks $1, 2, \dots, k$, so

that both the adversary and the deterministic online algorithm place these blocks into the cache. The next request is for block $k + 1$. In order to make room in the cache for block $k + 1$, the online algorithm evicts some block b_1 from the cache. The adversary, knowing that the online algorithm has just evicted block b_1 , makes the next request be for b_1 , so that the online algorithm must evict some other block b_2 to clear room in the cache for b_1 . As you might have guessed, the adversary makes the next request be for block b_2 , so that the online algorithm evicts some other block b_3 to make room for b_2 . The online algorithm and the adversary continue in this manner. The online algorithm incurs a cache miss on every request and therefore incurs n cache misses over the n requests.

Now let's consider an optimal offline algorithm, which knows the future. As discussed in Section 15.4, this algorithm is known as furthest-in-future, and it always evicts the block whose next request is furthest in the future. Since there are only $k + 1$ unique blocks, when furthest-in-future evicts a block, we know that it will not be accessed during at least the next k requests. Thus, after the first k cache misses, the optimal algorithm incurs a cache miss at most once every k requests. Therefore, the number of cache misses over n requests is at most $k + n/k$.

Since the deterministic online algorithm incurs n cache misses and the optimal offline algorithm incurs at most $k + n/k$ cache misses, the competitive ratio is at least

$$\frac{n}{k + n/k} = \frac{nk}{n + k^2}.$$

For $n \geq k^2$, the above expression is at least

$$\frac{nk}{n + k^2} \geq \frac{nk}{2n} = \frac{k}{2}.$$

Thus, for sufficiently long request sequences, we have shown the following:

Theorem 27.4

Any deterministic online algorithm for caching with a cache size of k has competitive ratio $\Omega(k)$. ■

Although we can analyze the common caching strategies from the point of view of competitive analysis, the results are somewhat unsatisfying. Yes, we can distinguish between algorithms with a competitive ratio of $\Theta(k)$ and those with unbounded competitive ratios. In the end, however, all of these competitive ratios are rather high. The online algorithms we have seen so far are deterministic, and it is this property that the adversary is able to exploit.

27.3.2 Randomized caching algorithms

If we don't limit ourselves to deterministic online algorithms, we can use randomization to develop an online caching algorithm with a significantly smaller competitive ratio. Before describing the algorithm, let's discuss randomization in online algorithms in general. Recall that we analyze online algorithms with respect to an adversary who knows the online algorithm and can design requests knowing the decisions made by the online algorithm. With randomization, we must ask whether the adversary also knows the random choices made by the online algorithm. An adversary who does not know the random choices is *oblivious*, and an adversary who knows the random choices is *nonoblivious*. Ideally, we prefer to design algorithms against a nonoblivious adversary, as this adversary is stronger than an oblivious one. Unfortunately, a nonoblivious adversary mitigates much of the power of randomness, as an adversary who knows the outcome of random choices typically can act as if the online algorithm is deterministic. The oblivious adversary, on the other hand, does not know the random choices of the online algorithm, and that is the adversary we typically use.

As a simple illustration of the difference between an oblivious and nonoblivious adversary, imagine that you are flipping a fair coin n times, and the adversary wants to know how many heads you flipped. A nonoblivious adversary knows, after each flip, whether the coin came up heads or tails, and hence knows how many heads you flipped. An oblivious adversary, on the other hand, knows only that you are flipping a fair coin n times. The oblivious adversary, therefore, can reason that the number of heads follows a binomial distribution, so that the expected number of heads is $n/2$ (by equation (C.41) on page 1199) and the variance is $n/4$ (by equation (C.44) on page 1200). But the oblivious adversary has no way of knowing exactly how many heads you actually flipped.

Let's return to caching. We'll start with a deterministic algorithm and then randomize it. The algorithm we'll use is an approximation of LRU called MARKING. Rather than "least recently used," think of MARKING as simply "recently used." MARKING maintains a 1-bit attribute *mark* for each block in the cache. Initially, all blocks in the cache are unmarked. When a block is requested, if it is already in the cache, it is marked. If the request is a cache miss, MARKING checks to see whether there are any unmarked blocks in the cache. If all blocks are marked, then they are all changed to unmarked. Now, regardless of whether all blocks in the cache were marked when the request occurred, there is at least one unmarked block in the cache, and so an arbitrary unmarked block is evicted, and the requested block is placed into the cache and marked.

How should the block to evict from among the unmarked blocks in the cache be chosen? The procedure RANDOMIZED-MARKING on the next page shows the

process when the block is chosen randomly. The procedure takes as input a block b being requested.

RANDOMIZED-MARKING(b)

```

1  if block  $b$  resides in the cache,
2       $b.mark = 1$ 
3  else
4      if all blocks  $b'$  in the cache have  $b'.mark = 1$ 
5          unmark all blocks  $b'$  in the cache, setting  $b'.mark = 0$ 
6          select an unmarked block  $u$  with  $u.mark = 0$  uniformly at random
7          evict block  $u$ 
8          place block  $b$  into the cache
9       $b.mark = 1$ 

```

For the purpose of analysis, we say that a new epoch begins immediately after each time line 5 executes. An epoch starts with no marked blocks in the cache. The first time a block is requested during an epoch, the number of marked blocks increases by 1, and any subsequent requests to that block do not change the number of marked blocks. Therefore, the number of marked blocks monotonically increases within an epoch. Under this view, epochs are the same as in the proof of Theorem 27.3: with a cache that holds k blocks, an epoch comprises requests for k distinct blocks (possibly fewer for the final epoch), and the next epoch begins upon a request for a block not in those k .

Because we are going to analyze a randomized algorithm, we will compute the expected competitive ratio. Recall that for an input I , we denote the solution value of an online algorithm A by $A(I)$ and the solution value of an optimal algorithm F by $F(I)$. Online algorithm A has an *expected competitive ratio* c if for all inputs I , we have

$$E[A(I)] \leq cF(I), \quad (27.12)$$

where the expectation is taken over the random choices made by A .

Although the deterministic MARKING algorithm has a competitive ratio of $\Theta(k)$ (Theorem 27.4 provides the lower bound and see Exercise 27.3-4 for the upper bound), RANDOMIZED-MARKING has a much smaller expected competitive ratio, namely $O(\lg k)$. The key to the improved competitive ratio is that the adversary cannot always make a request for a block that is not in the cache, since an oblivious adversary does not know which blocks are in the cache.